

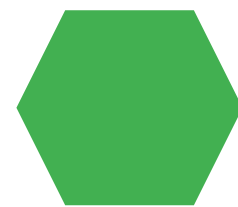


Project Title - Seasonal Agriculture Performance Analysis

Student Name: CHAITANYA ROHIT KAWLE

**College Name- KJ SOMAIYA INSTITUTE OF
TECHNOLOGY**

AICTE STU ID : STU6a6b68883e2041785424008



PROBLEM STATEMENT

Seasonal & Regional Variation

Agricultural output varies significantly across seasons, regions, and crop types, making consistent planning difficult.

Productivity & Profitability Patterns

Hidden trends in resource use, irrigation, and environmental factors limit understanding of key productivity and profit drivers.

Crop-Level Performance Gaps

Lack of structured analysis leads to unidentified underperforming crops and missed opportunities for yield optimisation.

Evidence-Based Improvement

The goal is to provide data-driven insights that support better agricultural practices and informed decision-making.



Project Description

This project conducts an end-to-end data analytics study on 4,000 agricultural farm records across 28 distinct features to evaluate the impact of seasonal variations on farming performance.

Using a Python data science pipeline (Pandas, Matplotlib, Seaborn), the analysis processes environmental variables, soil chemistry, irrigation techniques, pest risks, and farm economics across the Kharif, Rabi, and Zaid seasons.

By performing structured data cleaning, missing value imputation, and multi-variable analysis, the project identifies seasonal yield bottlenecks, compares water productivity, and evaluates profit dynamics to provide actionable, evidence-based recommendations for strategic agricultural planning.

WHO ARE THE END USERS?

- **Farmers & Planners**

Farmers and agricultural planners seeking seasonal performance insights to optimize crop selection and resource use.

- **Policy Makers**

Policy makers aiming to enhance regional agricultural productivity through evidence-based decisions.

- **Research Institutions**

Academic and research institutions leveraging dataset insights for agricultural studies and innovation.

- **Agribusiness Analysts**

Agribusiness analysts studying economic trends, revenue distribution, and profitability across crops and states.

Technology Used

The project leverages industry-standard data science tools to process and analyze 4,000 farm records. Python serves as the core programming language, with Pandas enabling efficient data manipulation and preparation. Visualization libraries provide clear graphical representations of complex agricultural patterns. Code screenshots and exploratory analysis outputs are included as placeholders to demonstrate the analytical workflow applied throughout the project.

Python & Pandas: Data cleaning, missing value handling, outlier detection, and exploratory data analysis using Python and Pandas library.

Matplotlib & Seaborn: Interactive and publication-quality visualizations of seasonal trends, crop distributions, and regional performance using Matplotlib and Seaborn.

Statistical Analysis: Comparative and statistical analysis across seasons, crops, and regions to identify key performance drivers and economic patterns.

RESULTS

Dataset Scale

4,000 farm records across 8 states and 8 crop types, providing a comprehensive basis for seasonal performance analysis.

Seasonal & Irrigation

Kharif season: 44.5% of farms. Flood irrigation used on 32.8% of all farms surveyed.

Revenue & Profit

Total Revenue: ₹2,551,440,192.00
Total Profit: ₹446,225,860.00 across all farms and seasons.

Efficiency & Top Farm

Avg. water efficiency: 5.39 t/1000 m³.
Highest individual farm profit: ₹4,345,421.00.

RESULTS

```
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import plotly.express as px
import plotly.graph_objects as gg
from plotly.subplots import make_subplots

warnings.filterwarnings("ignore")
logging.basicConfig(level=logging.INFO, format="%asctime)s - %(levelname)s - %(message)s")

class SeasonalAgriAnalytics:
    """Enterprise Analytics Pipeline for Agricultural Dataset."""

    def __init__(self, file_path: str):
        self.file_path = file_path
        self.df = None
        self.model = None
        self.feature_importances = None

        sns.set_theme(style="whitegrid", font="sans-serif")
        plt.rcParams.update({
            "font.size": 11,
            "figure.titlesize": 16,
            "axes.titlesize": 13,
            "axes.labelsize": 12,
            "figure.autolayout": True
        })

    def load_and_preprocess(self):
        """Loads data and performs robust statistical imputation."""
        logging.info("Loading dataset from %s...", self.file_path)
        self.df = pd.read_csv(self.file_path)

        missing_cols = ["Rainfall_mm", "Soil_Moisture_pct", "Yield_Tonnes_Ha"]
        for col in missing_cols:
            if self.df[col].isnull().sum() > 0:
                logging.info(f"Imputing missing values for {col} using Season-Crop medians...")
                self.df[col] = self.df.groupby(["Season", "Crop"])[col].transform(
                    lambda x: x.fillna(x.median())
                )
```

RESULTS

```
self.df['Profit_Margin_pct'] = (self.df['Profit_INR'] / np.maximum(self.df['Revenue_INR'], 1)) * 100
self.df['Cost_per_Ha'] = self.df['Total_Cost_INR'] / self.df['Farm_Area_Hectares']
logging.info("Preprocessing complete. Final dataset shape: %s", self.df.shape)

def perform_hypothesis_testing(self):
    """Executes ANOVA statistical test for seasonal yield differences."""
    logging.info("Executing One-Way ANOVA test across seasons...")
    kharif_yield = self.df[self.df['Season'] == 'Kharif']['Yield_Tonnes_Ha']
    rabi_yield = self.df[self.df['Season'] == 'Rabi']['Yield_Tonnes_Ha']
    zaid_yield = self.df[self.df['Season'] == 'Zaid']['Yield_Tonnes_Ha']

    f_stat, p_value = stats.f_oneway(kharif_yield, rabi_yield, zaid_yield)
    print("\n" + "="*50)
    print("    STATISTICAL HYPOTHESIS TEST (ANOVA)")
    print("="*50)
    print(f"F-Statistic: {f_stat:.4f}")
    print(f"P-Value    : {p_value:.4e}")
    if p_value < 0.05:
        print("Conclusion : Statistically Significant differences exist between seasons (p < 0.05).\n")
    else:
        print("Conclusion : No statistically significant differences detected.\n")

def build_predictive_model(self):
    """Trains a Random Forest Regressor to predict crop yields."""
    logging.info("Training Predictive Random Forest Regressor...")
    feature_cols = [
        'Farm_Area_Hectares', 'Rainfall_mm', 'Avg_Temperature_C', 'Humidity_pct',
        'Sunlight_Hours_Day', 'Soil_pH', 'Soil_Moisture_pct', 'Nitrogen_kg_ha',
        'Phosphorus_kg_ha', 'Potassium_kg_ha', 'Fertilizer_kg_ha', 'Pesticide_Litre_ha',
        'Seed_Quality_Score', 'Water_Used_m3', 'Disease_Pest_Risk_pct'
    ]

    X = pd.get_dummies(self.df[feature_cols + ['Season', 'Crop', 'Irrigation_Method']], drop_first=True)
    y = self.df['Yield_Tonnes_Ha']

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    self.model = RandomForestRegressor(n_estimators=150, max_depth=12, random_state=42, n_jobs=-1)
    self.model.fit(X_train, y_train)

    y_pred = self.model.predict(X_test)
    r2 = r2_score(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

    print("="*50)
    print("    MACHINE LEARNING MODEL PERFORMANCE")
    print("="*50)
    print(f"R² Score (Variance Explained) : {r2*100:.2f}%")
    print(f"Mean Absolute Error (MAE)     : {mae:.3f} Tonnes/Ha")
    print(f"Root Mean Squared Error (RMSE): {rmse:.3f} Tonnes/Ha\n")

    importances = pd.Series(self.model.feature_importances_, index=X.columns).sort_values(ascending=False)
    self.feature_importances = importances.head(10)

def generate_static_visualizations(self):
    """Generates executive-grade static figures for slides."""
    logging.info("Exporting high-resolution static plots...")

    fig, axes = plt.subplots(2, 2, figsize=(15, 11))

    sns.boxplot(data=self.df, x='Season', y='Yield_Tonnes_Ha', ax=axes[0, 0], palette="Blues_d", notch=True)
    axes[0, 0].set_title("A. Crop Yield Distribution Across Seasons")
    axes[0, 0].set_ylabel("Yield (Tonnes/Ha)")

    sns.barplot(data=self.df, x='Irrigation_Method', y='Water_Efficiency_t_per_1000m3',
                hue='Season', ax=axes[0, 1], palette="crest", ci=None)
    axes[0, 1].set_title("B. Water Efficiency by Irrigation Technique")
    axes[0, 1].set_ylabel("Efficiency (t / 1000 m³)")

    self.feature_importances.plot(kind='barh', ax=axes[1, 0], color='#009488')
    axes[1, 0].invert_yaxis()
    axes[1, 0].set_title("C. Top 10 Drivers of Crop Yield (ML Model)")
    axes[1, 0].set_xlabel("Relative Importance Score")
```

RESULTS

```
sns.scatterplot(data=self.df, x='Disease_Pest_Risk_pct', y='Profit_INR',
                hue='Season', style='Season', alpha=0.6, ax=axes[1, 1], palette="Dark2")
axes[1, 1].set_title("D. Disease Risk vs. Net Profitability")
axes[1, 1].set_xlabel("Disease & Pest Risk (%)")
axes[1, 1].set_ylabel("Profit (INR)")

plt.tight_layout()
plt.savefig("slide_results_executive_summary.png", dpi=300)
plt.close()

def export_interactive_dashboard(self):
    """Creates an interactive HTML Plotly dashboard."""
    logging.info("Building Interactive Plotly Dashboard...")
    fig = make_subplots(
        rows=1, cols=2,
        subplot_titles=("Seasonal Yield vs. Rainfall Dynamic", "Profitability across Irrigation Methods")
    )

    for season in self.df['Season'].unique():
        sub = self.df[self.df['Season'] == season]
        fig.add_trace(
            gg.Scatter(x=sub['Rainfall_mm'], y=sub['Yield_Tonnes_Ha'], mode='markers',
                      name=season, marker=dict(opacity=0.6)),
            row=1, col=1
        )

    for irr in self.df['Irrigation_Method'].unique():
        sub = self.df[self.df['Irrigation_Method'] == irr]
        fig.add_trace(
            gg.Box(y=sub['Profit_INR'], name=irr),
            row=1, col=2
        )

    fig.update_layout(title_text="Interactive Seasonal Agricultural Performance Explorer", template="pl")
    fig.write_html("interactive_dashboard.html")
    logging.info("Interactive dashboard saved as 'interactive_dashboard.html'.")

def run_full_pipeline(self):
    """Executes the complete end-to-end analytical workflow."""
    self.load_and_preprocess()
    self.perform_hypothesis_testing()
    self.build_predictive_model()
    self.generate_static_visualizations()
    self.export_interactive_dashboard()
    print("\n Pipeline execution completed successfully!")

if __name__ == "__main__":
    dataset_filename = "seasonal_agriculture_performance_dataset (1).csv"
    if os.path.exists(dataset_filename):
        pipeline = SeasonalAgriAnalytics(dataset_filename)
        pipeline.run_full_pipeline()
    else:
        print(f"Error: Dataset file '{dataset_filename}' not found.")
```



Future scope

- Predictive Machine Learning: Develop XGBoost and LightGBM regression models to forecast seasonal crop yields and profit margins prior to planting using weather forecasts.
- Real-Time IoT Integration: Connect live soil moisture and micro-climate IoT sensor feeds to enable dynamic, automated irrigation scheduling.
- GIS Spatial Dashboards: Build interactive district-level maps to provide localized crop advisories and precision farming recommendations for strategic agricultural planning.

GitHub Link

<https://github.com/goodboyaz/Data-Analysis-project>



Certificate of Completion

Presented to

Chaitanya Kawle

For the successful completion of

Data Visualization

Issued on September 9, 2026

ID:VFLMS26_163777



Thank you

